

Dynamic Batch Formation Simulator

Fixed, Dynamic, and Continuous Batching for LLM Inference

Team 16

Appaji Nagaraja Dheeraj (241CS110), team leader

Rishinandan D R (241CS145)

Ashlesh Prabhu (241CS214)

Sachin Rohra (241CS251)

Mentor: Balaji (balajink252is009@nitk.edu.in)

October 2026

1 Introduction

An LLM server receives requests at different times, and each request asks for a different number of output tokens. The server must decide which requests go to the GPU together. This decision is the batching strategy. It changes how much work the GPU does per step and how long each user waits.

We implemented three batching strategies and measured them with a real model on a real GPU. The strategies are fixed batching, dynamic batching, and continuous batching. We did not simulate token timings. Each run loads `Qwen/Qwen2.5-0.5B-Instruct` in FP16 and generates every token on an NVIDIA RTX GPU inside a Docker container.

We measured four things for each strategy:

- output-token throughput,
- request latency at p50 and p95,
- time to first token (TTFT) at p50 and p95,
- the average number of active requests per token step.

The source code, Docker files, raw results, and graphs are in the repository at <https://github.com/AppajiDheeraj/Dynamic-Batch-Formation-Simulator>.

2 Background

2.1 Why batching helps

A GPU can process several sequences in one tensor operation. For a small model, one token step reads all the model weights from GPU memory. If four sequences share that

step, the model reads the weights once for four tokens instead of once for one token. Batching therefore increases throughput. The cost is waiting time. If the server waits to collect a large batch, the first request in that batch waits too.

2.2 Request-level scheduling

In request-level scheduling, the server forms a batch and keeps it closed until every request in it finishes. Our fixed and dynamic strategies use this rule. A short request that finishes early leaves its slot empty. The slot stays empty until the longest request in the batch finishes, even if new requests are waiting.

2.3 Iteration-level scheduling

ORCA (OSDI 2022) introduced iteration-level scheduling for generative models [1]. The scheduler makes a new decision after every token step instead of after every batch. A finished request leaves at once, and a waiting request can take its slot at the next step. This idea is now called continuous batching. Our continuous strategy implements it.

vLLM adds PagedAttention to continuous batching [2]. PagedAttention stores the key-value (KV) cache in fixed-size blocks, so the engine wastes less GPU memory and can keep more sequences active. We did not use vLLM or copy its scheduler. We wrote one small inference engine and changed only the admission rule between tests. That way, any difference in the results comes from the scheduling rule.

3 System design

3.1 Architecture

Figure 1 shows the data flow. One process runs one strategy. The workload file supplies prompts and arrival times. The scheduler loop releases each request at its arrival time and asks the selected policy how many waiting requests to admit. The inference engine then generates one token for every active request.

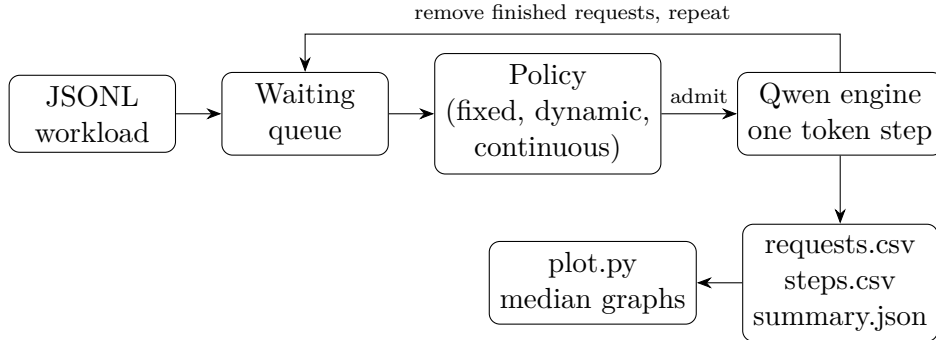


Figure 1: Benchmark data flow for one strategy run

3.2 Batching strategies

All three strategies use a maximum batch size of $B = 4$.

1. **Fixed batching.** The policy waits until four requests are in the queue. Then it starts a closed batch and runs it until all four requests finish. The last batch can be smaller if the workload has no more requests.
2. **Dynamic batching.** The policy starts a closed batch when four requests are in the queue or when the oldest waiting request has waited 50 ms. The batch then stays closed until all its requests finish.
3. **Continuous batching.** After every token step, the policy fills each free slot from the waiting queue. It admits $\min(B - \text{active}, \text{waiting})$ requests. It never waits on purpose.

Each policy is a short Python class in `src/batch_bench/schedulers.py`. The shared scheduler loop checks that no request is lost or counted twice after every admission and every step.

3.3 Inference engine

The engine in `src/batch_bench/model.py` left-pads the active sequences to the same length, builds an attention mask and position IDs, and runs one forward pass. It takes the most probable next token for each sequence (greedy decoding). A request finishes when it reaches its `max_new_tokens` limit or produces an end-of-sequence token. Before the timed run starts, the engine does a warm-up pass and resets the CUDA peak-memory counter.

3.4 Technology stack

Table 1: Software and model

Component	Version
Language	Python 3.11
Deep learning library	PyTorch 2.8.0 with CUDA 12.8
Model library	Hugging Face Transformers 4.46.3
Model	Qwen/Qwen2.5-0.5B-Instruct, FP16
Container	Docker 29.8.2 and Docker Compose, NVIDIA GPU runtime
Base image	pytorch/pytorch:2.8.0-cuda12.8-cudnn9-runtime
Graphs	Matplotlib 3.9.2

4 Workflow

One command runs the full experiment:

```
docker compose run --rm benchmark
```

Docker Compose starts `python -m batch_bench.experiment` inside the container. This script runs the dense workload first and the sparse workload second. For each workload, it calls `run.py` with three repetitions.

`run.py` starts a new child process for each strategy. The processes run one after the other, so two strategies never share the GPU. The strategy order rotates between repetitions (fixed, dynamic, continuous; then dynamic, continuous, fixed; and so on). This rotation reduces bias from GPU temperature or clock changes.

Each child process loads the model, does the warm-up pass, reads the workload, and runs the scheduler loop until all requests finish. It then writes three files:

- `requests.csv`, one row per request with arrival, start, first-token, and finish times,
- `steps.csv`, one row per token step with the active batch size,
- `summary.json`, the percentiles, throughput, and peak CUDA memory for the run.

After each child exits, `run.py` checks that the result contains every request ID from the workload exactly once. When all runs finish, `plot.py` reads the three repetitions and draws the graphs.

5 Experimental setup

5.1 Hardware

- **GPU:** NVIDIA GeForce RTX 5050 Laptop GPU, 8151 MiB, driver 592.82.
- **CPU:** 13th Gen Intel Core i7-13620H.
- **RAM:** 23.6 GiB.
- **Operating system:** Windows 11 Home, Docker Desktop with the NVIDIA runtime.
- **Power:** AC power, Windows “High performance” power plan.

The run happened on October 10, 2026. The model loaded from a local Hugging Face cache with `HF_HUB_OFFLINE=1`, so network speed did not affect the timings.

5.2 Workloads

Both workloads use the same 30 prompts and the same output limits. Only the gap between arrivals changes.

Table 2: Workloads

Workload	Requests	Arrival gap	Output limits (tokens)
Dense	30	20 ms	16, 32, 48
Sparse	30	400 ms	16, 32, 48

In the dense workload, requests arrive faster than the GPU finishes them, so a queue builds up. In the sparse workload, requests arrive slowly. This shows the cost of waiting for a full batch.

5.3 Metrics

Let a_i be the scheduled arrival time of request i , f_i the time of its first output token, and e_i the time of its last output token. Then:

$$\begin{aligned}\text{Latency}_i &= e_i - a_i \\ \text{TTFT}_i &= f_i - a_i \\ \text{Throughput} &= \frac{\sum_i \text{output tokens}_i}{\max_i e_i - \min_i a_i}\end{aligned}$$

Latency and TTFT start at the scheduled arrival, so both include time spent in the queue. The throughput denominator is the makespan of the whole run. In the sparse workload, it includes the planned gaps between arrivals.

Each strategy ran three times on each workload, which gives 18 runs. For every metric, we report the median of the three per-run values. Error bars on the graphs span the minimum and maximum run. Every run completed all 30 requests and produced exactly 960 output tokens, so all strategies did the same work.

5.4 Choice of dynamic wait limit

Before the main run, we did one run of each strategy at dynamic wait limits of 25, 50, 100, 200, and 400 ms. Table 3 shows the dynamic results.

Table 3: Dynamic batching wait-limit sweep (one run each)

Wait (ms)	Dense tok/s	Dense lat. p50	Sparse lat. p50	Sparse lat. p95
25	111.86	4483.39	1290.14	1890.20
50	109.32	4085.01	1237.79	1561.65
100	109.98	3947.56	1259.46	1722.86
200	104.87	4141.50	1573.81	1656.57
400	112.99	3909.86	1231.56	1904.19

Latency in milliseconds.

50 ms gave the lowest sparse p95 latency, so we used it for the main run. This sweep used the same workloads as the main run. It is tuning, not an independent test, and 50 ms may not be the best value for other workloads. An earlier full run at 25 ms is kept in `docs/results/windows-rtx-5050-2026-10-10/` for comparison.

6 Results

All numbers below are medians of three runs. Lower latency and TTFT are better. Higher throughput is better. The raw data for every run is in this repository folder:

`docs/results/windows-rtx-5050-2026-10-10-wait50/`

6.1 Dense workload

Table 4: Dense workload results

Strategy	Tok/s	Lat. p50	Lat. p95	TTFT p50	TTFT p95	Avg. batch
Fixed	111.89	3934.17	7387.23	3102.88	6557.84	2.50
Dynamic	113.84	4118.91	7310.49	3462.63	6759.23	2.50
Continuous	154.40	3082.77	5290.92	2150.59	4390.52	3.71

Latency and TTFT in milliseconds.

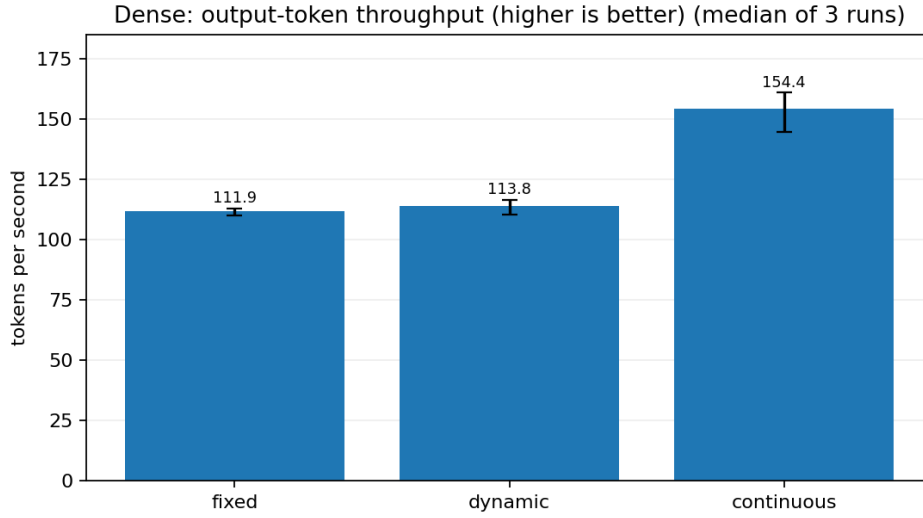


Figure 2: Dense workload: output-token throughput

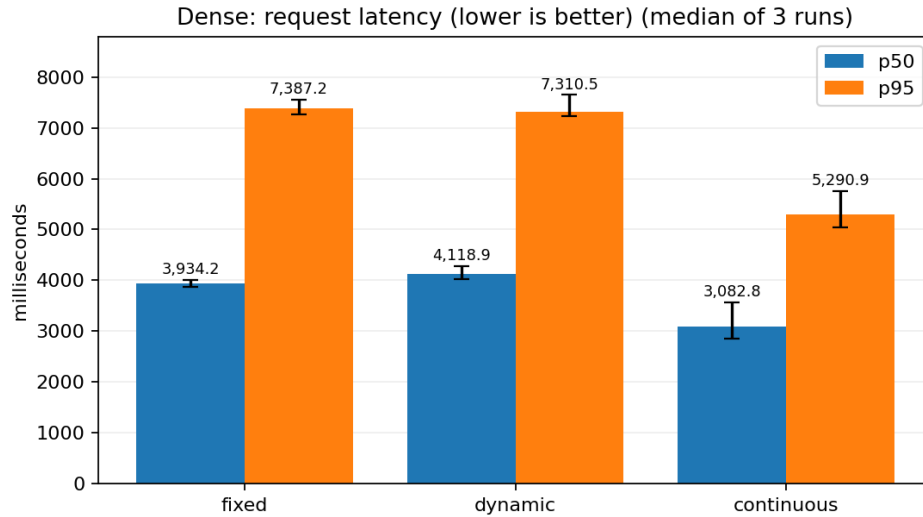


Figure 3: Dense workload: request latency

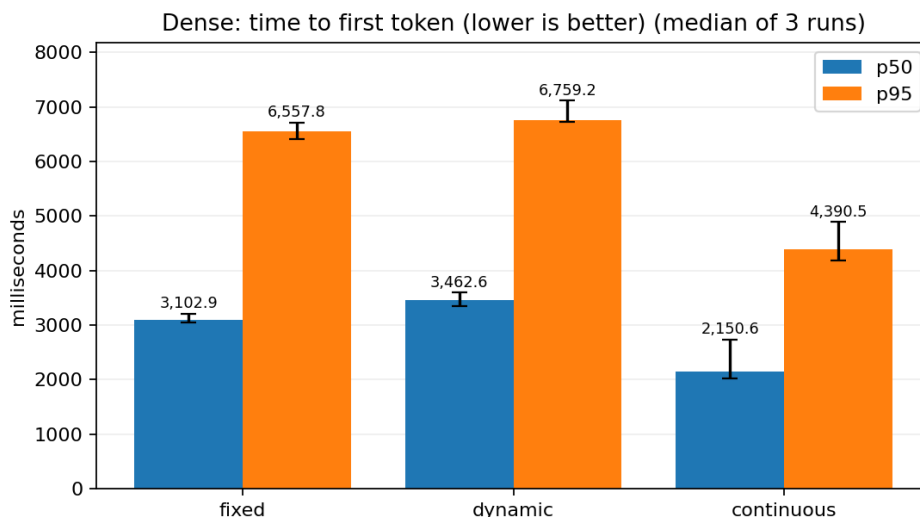


Figure 4: Dense workload: time to first token

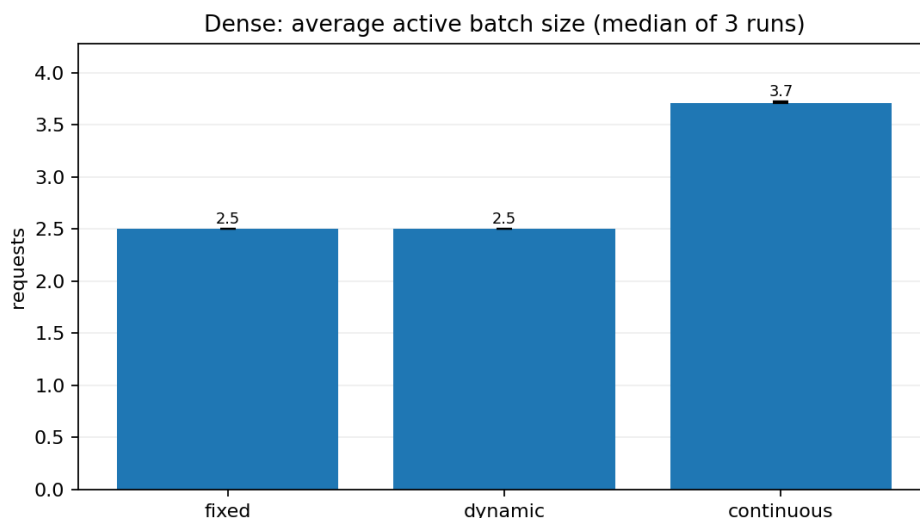


Figure 5: Dense workload: average active batch size

Continuous batching won every dense metric. Its throughput was 154.40 tokens/s against 111.89 for fixed, which is 38% higher. Its p95 latency was 28% lower and its p50 TTFT was 31% lower.

The batch-size graph explains why. Fixed and dynamic batches start with four requests, but the requests have output limits of 16, 32, and 48 tokens. The short requests finish first and their slots stay empty until the longest one ends. The average active batch therefore drops to 2.50. Continuous batching refills a slot on the next step, so its average stayed at 3.71 out of 4. More active requests per step means more tokens for each forward pass.

Dynamic batching did not help under dense arrivals. While the GPU is busy, the queue grows, so four or more requests are usually waiting when a batch ends. The 50 ms deadline seldom fires and dynamic acts almost like fixed. Its median TTFT was 12% worse than fixed. When the deadline does fire, dynamic starts a smaller closed batch, and the requests that arrive just after must wait for that whole batch to finish.

6.2 Sparse workload

Table 5: Sparse workload results

Strategy	Tok/s	Lat. p50	Lat. p95	TTFT p50	TTFT p95	Avg. batch
Fixed	73.91	1506.56	2184.72	586.26	1226.80	2.50
Dynamic	73.42	1256.70	1590.78	477.85	993.06	1.62
Continuous	75.94	736.47	1151.14	34.53	45.51	1.76

Latency and TTFT in milliseconds.

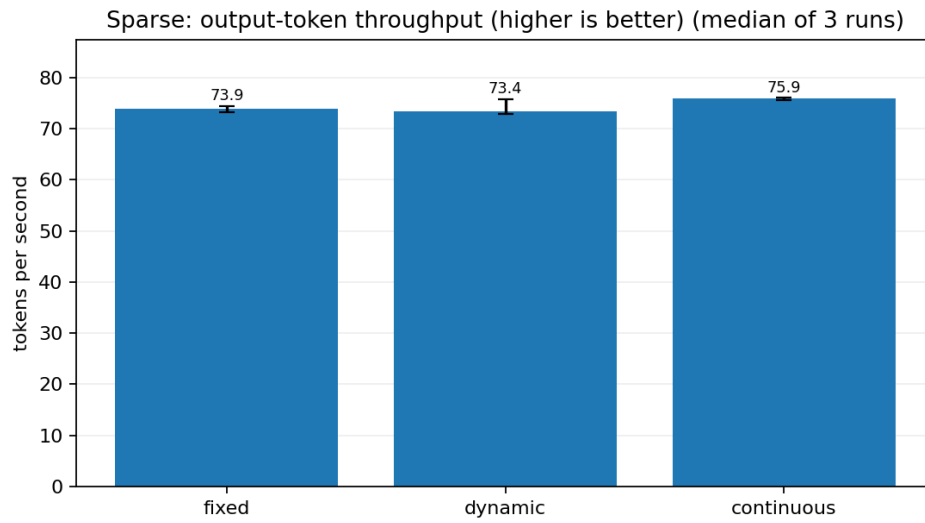


Figure 6: Sparse workload: output-token throughput

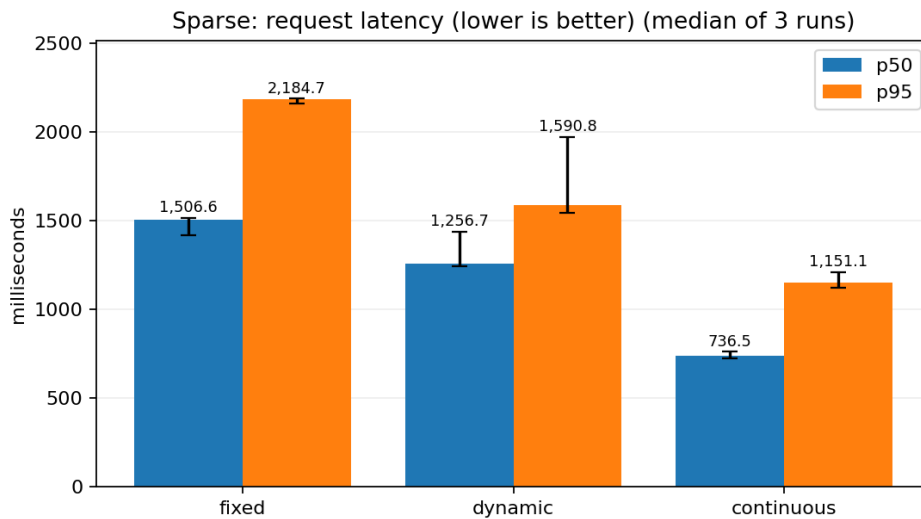


Figure 7: Sparse workload: request latency

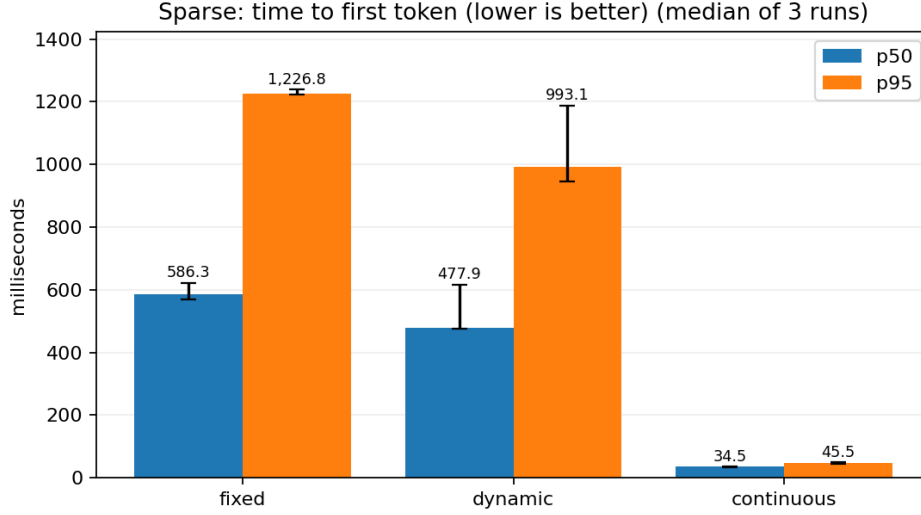


Figure 8: Sparse workload: time to first token

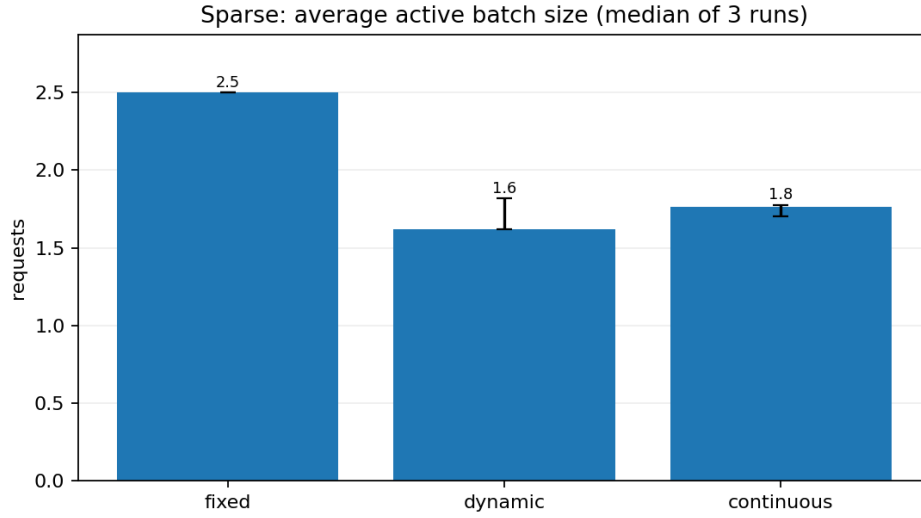


Figure 9: Sparse workload: average active batch size

Throughput was almost equal for all three strategies, between 73.42 and 75.94 tokens/s. The arrival schedule sets this number, not the scheduler. The last request arrives at 11.6 s, and the median makespan of each strategy was between 12.6 s and 13.1 s.

Latency is where the strategies differ. Fixed batching waits for four requests. At a 400 ms gap, that wait is up to 1.2 s, and the TTFT graph shows it. Dynamic batching cuts the wait at 50 ms. Compared with fixed, it lowered p50 latency by 17%, p95 latency by 27%, p50 TTFT by 18%, and p95 TTFT by 19%. Continuous batching admits each request on the next step after it arrives. Its p50 TTFT was 34.53 ms, which is 17 times lower than fixed. Its p50 latency was 736.47 ms, about half of the fixed value.

The continuous TTFT bars in Figure 8 are small but not zero. They look flat because the same axis also shows fixed and dynamic values above 900 ms. The numbers on the bars give the exact values.

The average batch size of continuous (1.76) is smaller than fixed (2.50) in this workload. This is the expected trade. Fixed fills its batches by making requests wait. Con-

tinuous starts work at once with fewer requests on the GPU, and the arrival rate is too low to fill more slots.

6.3 GPU memory

Table 6 gives the median peak CUDA memory that PyTorch allocated after warm-up.

Table 6: Peak allocated CUDA memory (MiB, median of three runs)

Workload	Fixed	Dynamic	Continuous
Dense	993.29	993.29	1023.61
Sparse	993.29	982.44	990.95

Every peak stayed near 1 GiB. Continuous batching used 30 MiB more under dense load because it kept more sequences active. This PyTorch counter does not include the CUDA context or driver memory. Even with those added, the benchmark fits the 3 to 4 GB VRAM target for this assignment.

7 Limitations

The engine runs real Qwen inference, but it calls the model with `use_cache=False`. At each token step, it recomputes attention over the full prompt and all earlier output tokens. A production server keeps a KV cache and computes only the new token.

We chose this on purpose. All three strategies use the same inference path, so no strategy gets a faster model or a different memory policy. The difference in the results comes only from the admission rule. The cost is that each step does more work than necessary, so absolute latency and throughput are worse than a real server would give.

With a KV cache, every step would be faster, and all the absolute numbers would change. Cache memory would also grow with the number of active sequences, so memory would become part of the scheduling decision. We expect continuous batching to keep its lead, because the slot-refill effect does not depend on the step cost. The size of the gaps would change, though, and we did not measure it.

Other limits:

- We tested one model, one GPU, one batch size ($B = 4$), and two arrival patterns.
- The 50 ms dynamic wait limit was tuned on the same workloads we report.
- Three repetitions show the spread between runs but are not enough for strong statistical tests.
- We did not implement PagedAttention, preemption, or multi-GPU execution.

8 Conclusion

Continuous batching gave the best result on both workloads in our setup. Under dense load, it raised throughput by 38% because it kept 3.71 of 4 slots busy, against 2.50 for

the closed-batch strategies. Under sparse load, it cut the median time to first token from 586 ms to 35 ms because no request waited for a batch to fill.

Dynamic batching is a partial fix. Its deadline helped under sparse arrivals, where it cut p95 latency by 27% compared with fixed. Under dense arrivals, it behaved like fixed batching because it still keeps each batch closed until the longest request ends.

The main lesson for us is that the closed batch is the real problem, not the batch size. Fixed and dynamic batching differ only in when a batch starts. Both waste slots after short requests finish. ORCA’s step-level admission removes that waste, and our measurements on a 0.5B model show the effect clearly even at batch size 4.

References

- [1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. ORCA: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022. <https://www.usenix.org/conference/osdi22/presentation/yu>
- [2] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP 2023*. Source: <https://github.com/vllm-project/vllm>
- [3] Qwen Team. Qwen2.5-0.5B-Instruct model card. <https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct>
- [4] Hugging Face. Transformers documentation. <https://huggingface.co/docs/transformers>
- [5] PyTorch documentation. <https://pytorch.org/docs/stable/>
- [6] Docker documentation. <https://docs.docker.com/>